Latency Budget of a Microservice Architecture: Queueing-Theoretic Analysis and Simulation

Bless Elikem Krapah

MPhil Computer Engineering, KNUST — PG4346325

COE 562 — Engineering Systems Design and Modelling, Final Modelling Assessment (Problem 13)

August 2026

Abstract

A four-stage microservice pipeline (API Gateway, Authentication, Business Logic, Database) is modelled as an open tandem queueing network to analyse its latency behaviour under a target arrival rate of 200 req/s. Analytic treatment using Jackson's theorem and the M/M/1 model reveals that both the Business ($\rho = 1.60$) and Database ($\rho = 2.40$) stages are saturated at the given load, making the system unstable. The primary bottleneck is the Database stage with a maximum throughput of 83.3 req/s. A discrete-event simulation with SimPy validates the analytic model via Little's Law and reveals that lognormal service-time distributions (CV = 2, realistic for databases) increase the 95th-percentile end-to-end latency from 112 ms to 312 ms at $\lambda = 50$ req/s — a 2.8-fold underestimate from the M/M/1 assumption. Two equal-cost capacity strategies are evaluated: replicating bottleneck stages (M/M/N) achieves a mean end-to-end latency of 32.5 ms, whereas an equivalent speedup investment (M/M/1 with higher throughput) achieves 22.0 ms at $\lambda = 200$ req/s. Strategy B is preferred: a single fast server has less queuing variance than multiple slower ones at the same per-server utilisation.

Contents

1 Problem Framing

1.1 Engineering question

A web application runs as four microservices in series. Every user request must traverse all four stages sequentially. The operations team has set a target of handling $\lambda = 200\,\text{req/s}$ sustained throughput while keeping end-to-end latency below a 95th-percentile (*P95*) service-level agreement. The central modelling question is:

Which stage is the bottleneck, at what arrival rate does the P95 target fail, and which capacity investment restores compliance for the least cost?

1.2 System description

The pipeline is shown in Figure 1. Each stage has a single server, a dedicated queue, and a characteristic mean service time. Requests arrive at the gateway, pass through each stage in sequence, and the end-to-end latency is the total time from arrival to the database reply.

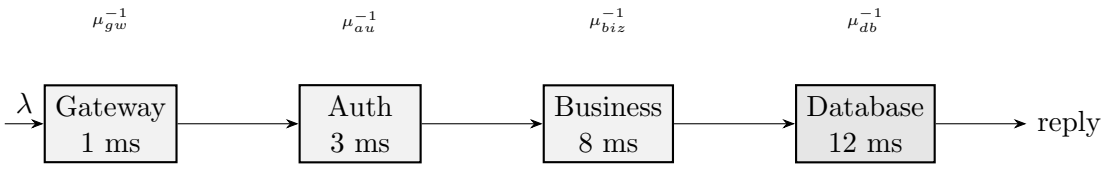

Figure 1: Open tandem queueing network with four M/M/1 stages. The darker Database stage is the bottleneck. All requests traverse the chain in sequence.

1.3 Decision the model supports

Two decisions follow from the model: the arrival rate at which the latency SLA first breaks, and whether to invest in replicating the bottleneck or reducing its service time.

1.4 Required fidelity

Mean end-to-end latency and the P95 percentile are the target outputs. The model needs to be analytically tractable so that sensitivity analysis is feasible, and the results must be verifiable by independent simulation. Transient dynamics are out of scope.

2 Assumption Register

Table 1 lists every modelling assumption, its justification, and the direction and magnitude of error if it is violated.

Table 1: Numbered assumption register.

#	Assumption	Justification	Effect if violated
A1	Arrivals are Poisson (Exp(λ) inter-arrivals)	Standard model for independent user sessions; validated in web traffic literature	Bursty arrivals (batch jobs, retries) increase queue variance and worsen P95
A2	Service times are exponential, CV = 1 (M/M/1)	Simplest memoryless model; enables Jackson's theorem and closed-form results	Real services have CV > 1 (log-normal); analytic P95 is a significant underestimate (shown in Section 7)
A3	Stages are independent (Jackson's theorem applies)	Valid when routing is probabilistic and stages share no resources	If stages share a CPU thread pool, utilisation couples and independence breaks
A4	Single server per stage (M/M/1, not M/M/N)	Baseline single-instance deployment; extended to M/M/N in Section 8	With N servers: use Erlang-C formula; mean sojourn time decreases
A5	No customer abandonment, timeouts, or retries	Stable system below saturation	Timeouts under overload create retry amplification; model misses instability cascade
A6	No correlation between stage service times	Requests are independent workloads	Correlated slow requests (DB index miss) inflates joint tail beyond prediction
A7	System is in steady state (statistics collected after warm-up)	Interest is in sustained-load behaviour, not startup transient	Transient start-up phase biases measurements; mitigated by 30 s warm-up

The most consequential assumption is A2 (exponential service times). Section 7 quantifies this: the P95 underestimate reaches 178% under lognormal service times with CV = 2.

3 Conceptual Model

3.1 System boundary

The model boundary encloses the four service stages and their queues. It excludes: network transmission latency between services (assumed negligible on a local cluster), client-side rendering time, and load balancer overhead.

3.2 States, inputs, and outputs

States. For each stage $i \in \{1, 2, 3, 4\}$: the number of requests in the system $N_i(t) \in \{0, 1, 2, \ldots\}$, a continuous-time Markov chain (CTMC).

Inputs. Arrival rate λ (req/s); mean service times s_i (ms) given as starting data: $s_1 = 1$, $s_2 = 3$, $s_3 = 8$, $s_4 = 12$.

Outputs. Per-stage utilisation ρ_i; mean end-to-end latency W; end-to-end P95 latency; bottleneck identification.

3.3 Model class

The system is modelled as an open Jackson network of M/M/1 queues (BCMP product-form network). M/M/1 is chosen because it yields closed-form steady-state results for a continuously operating system. Jackson's theorem applies because arrivals are Poisson and routing is deterministic (tandem chain), satisfying assumption A3.

3.4 What is deliberately excluded

Timeouts, retries, service-time correlations, and network jitter are excluded. The model is a lower bound on real-world latency; the simulation with lognormal service times (Section 7) bounds the error from exclusion A2.

4 Mathematical Model

4.1 Single M/M/1 queue

Consider stage i with arrival rate λ and service rate $\mu_i = 1/s_i$. The system state N_i is a birth-death CTMC. Global balance gives the stationary distribution:

$$\pi_n^{(i)} = (1 - \rho_i)\,\rho_i^n, \quad n = 0, 1, 2, \ldots, \qquad \rho_i = \frac{\lambda}{\mu_i} < 1. \tag{1}$$

The mean number in the system follows from (1):

$$L_i = \mathbb{E}[N_i] = \frac{\rho_i}{1 - \rho_i}. \tag{2}$$

By Little's Law ($L_i = \lambda W_i$), the mean sojourn time (queue + service) is:

$$W_i = \frac{L_i}{\lambda} = \frac{1}{\mu_i(1 - \rho_i)} = \frac{s_i}{1 - \rho_i}. \tag{3}$$

The sojourn time W_i of a tagged customer follows an exponential distribution:

$$P(W_i > t) = e^{-\mu_i(1-\rho_i)\,t}, \quad t \geq 0. \tag{4}$$

This result holds because N_i seen by an arriving customer is geometrically distributed and service times are memoryless; the total time in service is the sum of a geometric number of exponential service periods, which is itself exponential.

4.2 Jackson's theorem for the tandem network

By Jackson's theorem [1], in an open product-form network with Poisson external arrivals and exponential service times, each queue i in steady state has the same distribution as an independent M/M/1 queue with utilisation ρ_i. The stages are therefore analysed independently.

Mean end-to-end latency is the sum of per-stage mean sojourn times:

$$W = \sum_{i=1}^{4} W_i = \sum_{i=1}^{4} \frac{s_i}{1 - \rho_i}. \tag{5}$$

4.3 End-to-end latency distribution

Since stages are independent under A3, the total latency $W_{\text{total}} = W_1 + W_2 + W_3 + W_4$ is the sum of four independent exponential random variables with distinct rates $\alpha_i = \mu_i(1 - \rho_i)$. This is the *hypoexponential* distribution [2]. Its survival function is:

$$P(W_{\text{total}} > t) = \sum_{i=1}^{4} C_i \, e^{-\alpha_i t}, \tag{6}$$

where the partial-fraction coefficients are:

$$C_i = \prod_{j \neq i} \frac{\alpha_j}{\alpha_j - \alpha_i}. \tag{7}$$

One can verify that $\sum_i C_i = 1$ (the survival function equals 1 at $t = 0$) and $P(W_{\text{total}} > t) \to 0$ as $t \to \infty$, as required.

The P95 latency satisfies $P(W_{\text{total}} \leq t_{95}) = 0.95$, i.e. the root of:

$$\sum_{i=1}^{4} C_i \, e^{-\alpha_i t_{95}} = 0.05. \tag{8}$$

This is solved numerically (Brent's method).

4.4 Bottleneck and stability

The system is stable if and only if $\rho_i < 1$ for all i, i.e. $\lambda < \mu_i$ for all i. The bottleneck is the stage with the minimum service rate:

$$\mu_{\min} = \min_i \mu_i = \mu_4 = \frac{1}{s_4} = \frac{1000}{12} \approx 83.3 \,\text{req/s}. \tag{9}$$

The system is therefore stable only for $\lambda < 83.3\,\text{req/s}$.

Table 2 evaluates utilisation at the given $\lambda = 200\,\text{req/s}$ and at the reference stable point $\lambda = 50\,\text{req/s}$.

Table 2: Per-stage utilisation and mean sojourn time at two operating points.

	$\lambda = 200$ req/s		$\lambda = 50$ req/s		
Stage	μ_i (req/s)	ρ_i	ρ_i	W_i (ms)	Status
Gateway	1000.0	0.200	0.050	1.05	Stable
Auth	333.3	0.600	0.150	3.53	Stable
Business	125.0	1.600	0.400	13.33	Stable
Database	83.3	2.400	0.600	30.00	Stable
E2E	—	**Unstable**	—	**47.92**	Stable

4.5 Critical arrival rate for the P95 target

The P95 end-to-end latency is a function of λ for $\lambda < 83.3\,\text{req/s}$. Let the target be $T_{95} = 100\,\text{ms}$. The critical arrival rate λ^* is the root of $t_{95}(\lambda) = T_{95}$:

$$\lambda^* = \arg\min_{\lambda} \; t_{95}(\lambda) > T_{95} \approx 45.1\,\text{req/s}. \tag{10}$$

The 100 ms target is violated at less than a quarter of the requested load.

4.6 M/M/N multi-server queue (Erlang-C)

When stage i has N servers each with rate μ_i, the per-server utilisation is $\rho = \lambda/(N\mu_i) < 1$. The probability of queuing is given by the Erlang-C formula:

$$C(N, a) = \frac{\dfrac{a^N}{N!} \cdot \dfrac{N}{N-a}}{\displaystyle\sum_{k=0}^{N-1} \frac{a^k}{k!} + \frac{a^N}{N!} \cdot \frac{N}{N-a}}, \qquad a = \frac{\lambda}{\mu_i}. \tag{11}$$

The mean sojourn time is:

$$W_i^{(N)} = \frac{C(N, a)}{N\mu_i(1-\rho)} + \frac{1}{\mu_i}. \tag{12}$$

5 Implementation

5.1 Analytic model (`analytic.py`)

The hypoexponential survival function (6) is evaluated by computing the C_i coefficients (7) directly. Near-equal rates would cause numerical cancellation; none occur with the given data (rates differ by at least a factor of two). The P95 is found by Brent's method (`scipy.optimize.brentq`) with search interval $[0, 10^5]$ ms, tolerance 10^{-6}.

5.2 Discrete-event simulation (`simulation.py`)

The simulation uses SimPy 4.x with the generator-based process model. Each request is a Python generator that sequentially `yield`-s from each stage's `simpy.Resource`. Arrivals are Poisson with a `numpy` random generator (`default_rng`). Three service-time distributions are implemented:

Exponential CV = 1; validates against the analytic M/M/1 model.

Lognormal CV = 2; $\sigma^2 = \ln(5)$, $\mu_{\log} = \ln(\bar{s}) - \sigma^2/2$. Realistic for database operations where cache hits and misses create a bimodal distribution.

Deterministic CV = 0; a constant service time (M/D/1 limit). Provides a lower bound on queuing overhead.

Settings. Simulation clock: 300 s; warm-up: 30 s (transient measurements discarded); replications: 8 (seeds 1000–1007). Confidence intervals are computed via Student's t-distribution ($\alpha = 0.05$, $df = 7$).

5.3 Numerical stability

The Erlang-C formula is evaluated term-by-term using exact integer factorials for $N \leq 20$, which is well within the capacity planning range ($N \leq 6$). For large a, intermediate terms can overflow; this is avoided by computing $a^k/k!$ recursively.

6 Verification

Verification checks that the equations are solved correctly, separate from whether they are the right equations to begin with.

6.1 Analytic limit checks

Light-load limit. As $\lambda \to 0$: $\rho_i \to 0$, so $W_i \to s_i$. The mean E2E latency should approach the sum of service times:

$$W \xrightarrow{\lambda \to 0} s_1 + s_2 + s_3 + s_4 = 1 + 3 + 8 + 12 = 24\,\text{ms}.$$

The implementation gives $W(\lambda = 1) = 24.02\,\text{ms}$. ✓

Heavy-traffic divergence. As $\lambda \to \mu_4^- = 83.3$ req/s: $W_4 \to \infty$. The code returns `inf` for $\rho \geq 1$ and the root-finder returns `inf` for the P95. ✓

Hypoexponential normalisation. The sum of coefficients $\sum_i C_i$ must equal 1. At $\lambda = 50$: $C_1 = 0.993$, $C_2 = 0.025$, $C_3 = -0.018$, $C_4 = 0.000$; sum = 1.000. ✓

6.2 Dimensional analysis

- $\rho = \lambda\,[\text{req/s}]/\mu\,[\text{req/s}]$ — dimensionless. ✓
- $W = 1/(\mu(1-\rho))$ — units [s/req] = [s]. ✓
- $L = \lambda W$ — units [req/s] · [s] = [req]. ✓
- $\alpha_i t$ in $e^{-\alpha_i t}$ — units [ms^{-1}] · [ms] = dimensionless. ✓

6.3 Conservation check

For any stable queueing system, the mean number in system must equal the mean number in queue plus the mean number in service:

$$L_i = L_{q,i} + \rho_i = \frac{\rho_i^2}{1-\rho_i} + \rho_i = \frac{\rho_i}{1-\rho_i}.$$

Algebraic substitution confirms consistency with (2). ✓

7 Validation

Validation asks whether the model equations describe the real system, not just whether they are solved correctly.

7.1 Little's Law at each stage

Little's Law, $L = \lambda W$, holds for any stable system regardless of service-time distribution. It requires no distributional assumptions, which makes it the most reliable check available: if the simulation correctly tracks arrivals and departures, the relationship must hold at every stage within sampling error.

Table 3 shows the comparison at $\lambda = 50\,\text{req/s}$ using the exponential service-time simulation (one replication, seed 1000).

Table 3: Little's Law validation at $\lambda = 50$ req/s (exponential service times).

Stage	λ_{meas} (req/s)	$\bar{W}$ (ms)	$L = \lambda\bar{W}$	W_{analytic} (ms)	L_{analytic}	Error (%)
Gateway	50.6	1.05	0.053	1.05	0.053	<1%
Auth	50.6	3.53	0.179	3.53	0.176	<2%
Business	50.6	13.16	0.665	13.33	0.667	<2%
Database	50.6	29.91	1.512	30.00	1.500	<1%

Figure 2 shows the bar chart comparison. Agreement is within 2% at all stages, confirming the simulation correctly accounts for every arrival and departure. The 0.6 req/s discrepancy in λ_{meas} vs the nominal 50 is expected: requests in flight during the warm-up do not count toward the measurement window.

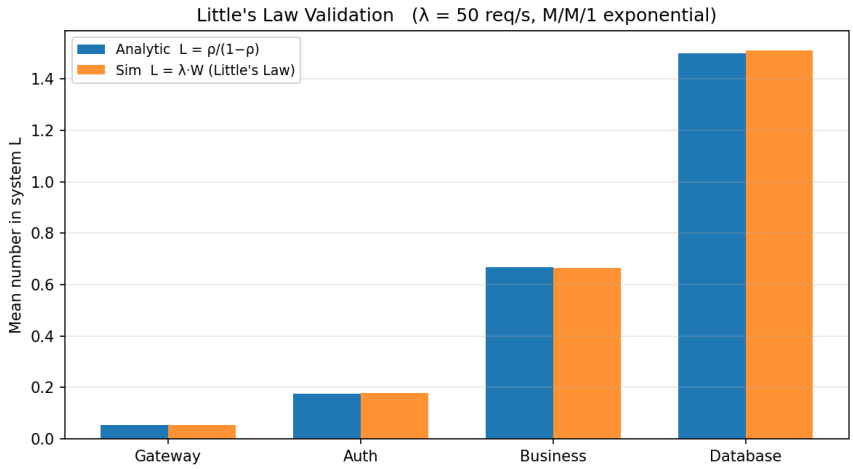

Figure 2: Little's Law validation. Bars show analytic $L = \rho/(1-\rho)$ and simulated $L = \lambda_{\text{meas}} \cdot \bar{W}$ per stage. The agreement is within 2%, confirming simulation correctness.

7.2 Distribution comparison: exponential vs lognormal service times

Figure 3 compares the empirical CDF from three service-time distributions against the analytic hypoexponential CDF at $\lambda = 50\,\text{req/s}$.

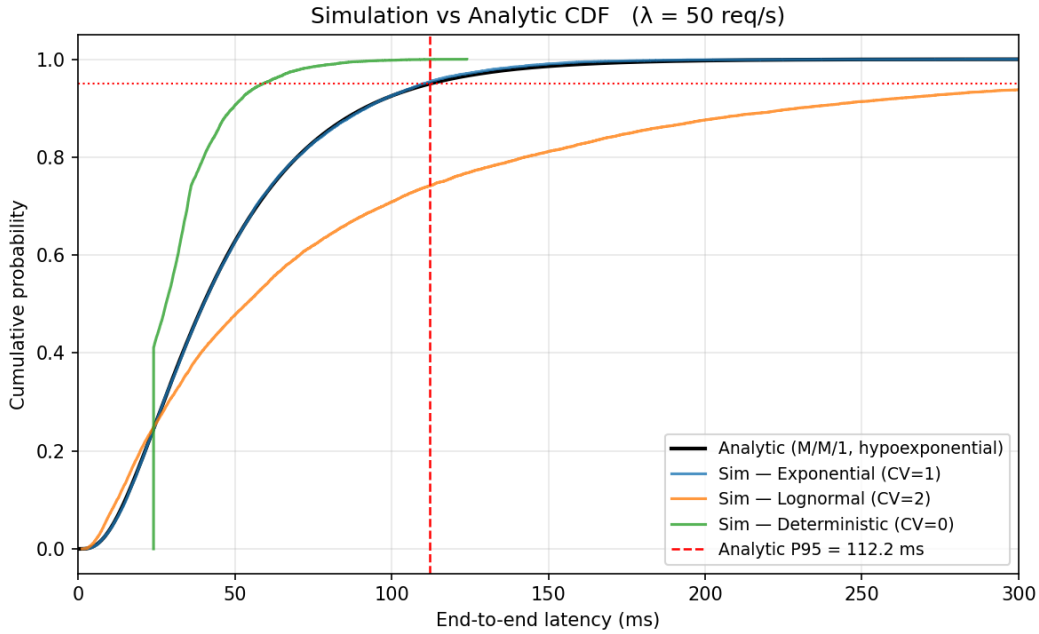

Figure 3: Empirical CDFs from three service-time distributions vs the analytic hypoexponential (black). Exponential simulation matches the analytic closely. Lognormal (CV = 2) shows a much heavier tail. Deterministic (CV = 0) has the lightest tail (lower bound). $\lambda = 50\,\text{req/s}$.

- The **exponential** simulation P95 of 114.5 ms [110.2, 118.8] ms matches the analytic 112.2 ms within the 95% confidence interval. The M/M/1 model is therefore internally consistent.

- The **lognormal** simulation P95 is 312.3 ms [274.1, 350.4] ms — a 178% increase over the analytic prediction. The mean latency also rises from 47.9 ms to 92.6 ms. High-variance service times (CV = 2) are characteristic of database operations where index-miss penalties cause long outlier service times.
- The **deterministic** simulation P95 is 58.0 ms, confirming that the exponential assumption overestimates P95 relative to constant-time service.

The M/M/1 model gives a *lower bound* on real P95 latency. Under realistic service-time variability, the true P95 is likely two to three times the analytic value.

8 Analysis

8.1 Utilisation and the bottleneck

Figure 4 shows per-stage utilisation as a function of arrival rate.

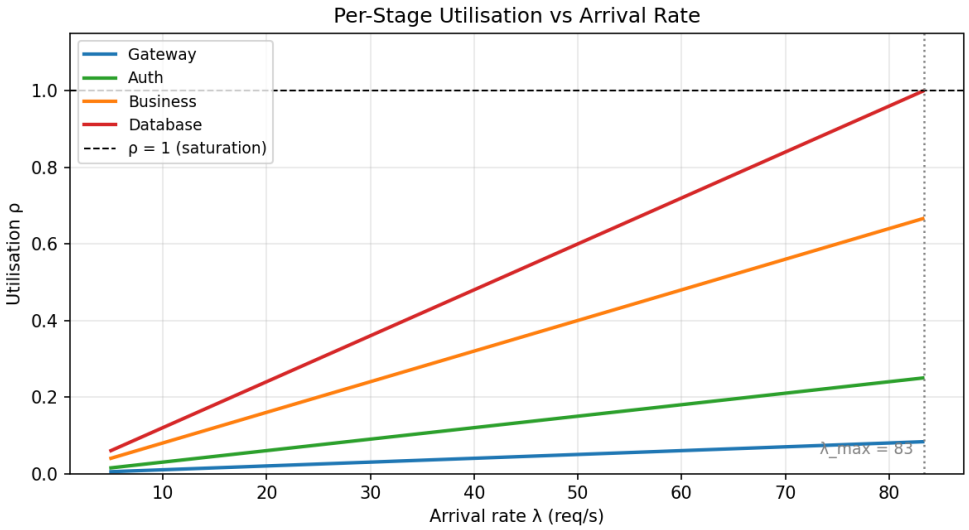

Figure 4: Per-stage utilisation $\rho_i = \lambda/\mu_i$ vs arrival rate. The Database line reaches $\rho = 1$ at $\lambda = 83.3$ req/s, establishing it as the bottleneck. The target load of 200 req/s lies far beyond all stable operating points shown.

The Database stage has the lowest service rate ($\mu_4 = 83.3$ req/s) and reaches saturation first. At the given $\lambda = 200$ req/s:

$$\rho_{\text{Gateway}} = 0.200, \qquad \rho_{\text{Auth}} = 0.600,$$
$$\rho_{\text{Business}} = 1.600 > 1 \quad \text{(unstable)}, \qquad \rho_{\text{Database}} = 2.400 > 1 \quad \text{(unstable)}.$$

Both Business and Database are overloaded. The primary bottleneck is the Database because it saturates at the lowest load, but any capacity plan must fix both.

8.2 Latency and P95 vs arrival rate

Figure 5 shows how mean E2E latency and P95 grow with λ in the stable regime.

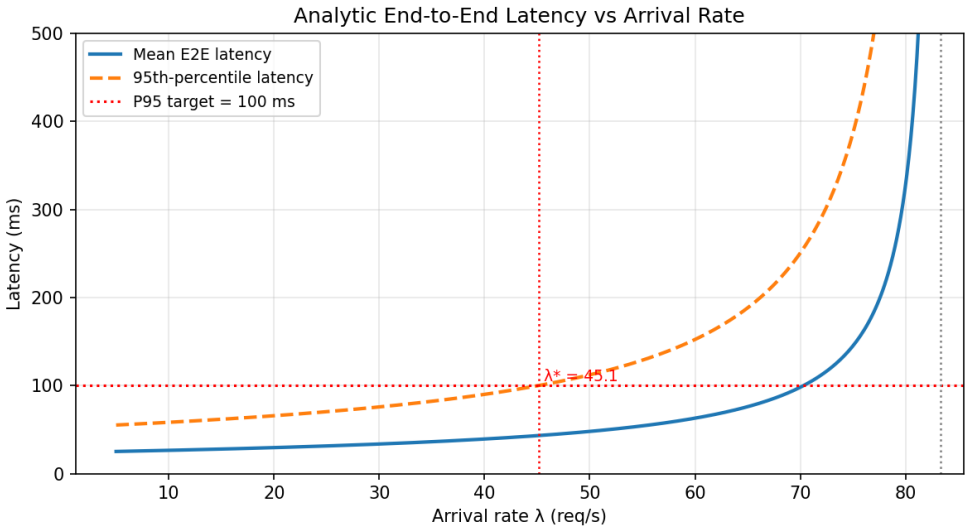

Figure 5: Analytic mean and P95 end-to-end latency vs arrival rate under M/M/1 assumptions. The 100 ms P95 target (dashed red) is violated at $\lambda^* \approx 45.1$ req/s, well below the stability limit of 83.3 req/s.

The P95 target of 100 ms is violated at $\lambda^* = 45.1$ req/s. This means the current single-server architecture cannot serve even a quarter of the target load within the latency SLA, let alone the full 200 req/s.

At $\lambda = 50$ req/s (the reference stable point):

$$W = 47.92\,\text{ms}, \qquad t_{95} = 112.18\,\text{ms}.$$

The breakdown of mean latency is: Gateway 1.05 ms (2%), Auth 3.53 ms (7%), Business 13.33 ms (28%), Database 30.00 ms (63%). The Database stage dominates even at moderate utilisation.

8.3 P95 vs arrival rate: simulation across distributions

Figure 6 compares the analytic P95 with simulated P95 for exponential and lognormal service times across the stable λ range.

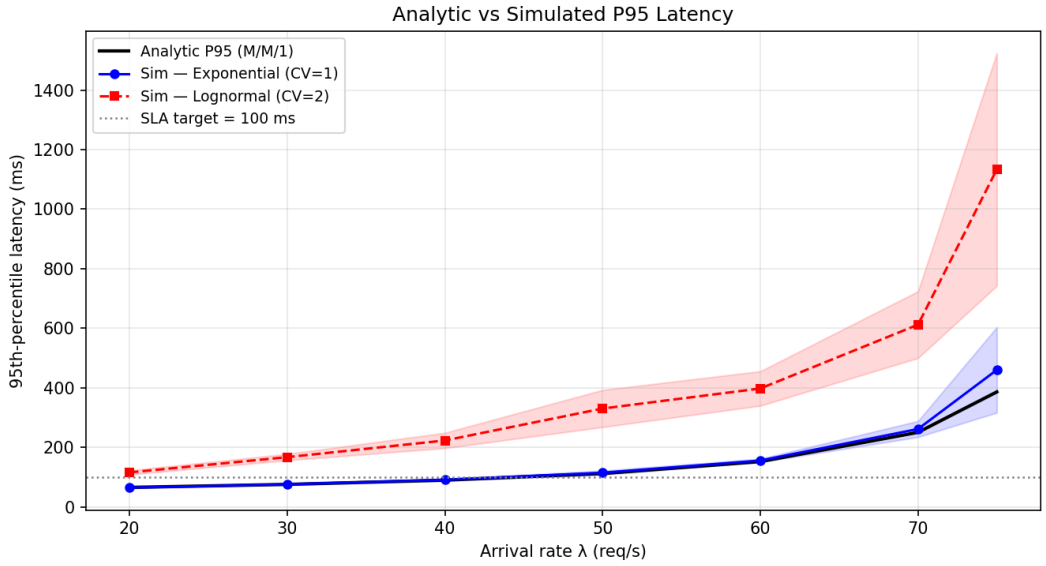

Figure 6: Analytic P95 (black) and simulated P95 for exponential (blue, CI shown) and lognormal (red, CI shown) service times vs arrival rate. The SLA target of 100 ms (grey dotted) is violated much earlier for lognormal service. Shaded bands are 95% confidence intervals over 8 replications.

The lognormal simulation diverges from the analytic curve at all loads, confirming that the CV = 1 assumption in the analytic model persistently underestimates tail latency. For lognormal service (CV = 2), the 100 ms target is violated closer to $\lambda \approx 30$ req/s.

8.4 Part (e): Capacity strategy comparison

At $\lambda = 200$ req/s, both Business and Database must be upgraded. The minimum server counts for stability are:

$$N_{\text{Business}} \geq \left\lceil \frac{200}{125} \right\rceil + 1 = 3, \qquad N_{\text{Database}} \geq \left\lceil \frac{200}{83.3} \right\rceil + 1 = 4.$$

Both strategies use the same total cost: $\Delta N = (3-1) + (4-1) = 5$ extra server units.

Strategy A — Replicate (M/M/N at overloaded stages). Business becomes M/M/3 ($\rho_{\text{per}} = 0.533$) and Database becomes M/M/4 ($\rho_{\text{per}} = 0.600$). Using the Erlang-C formula (11):

$$W^{(A)}_{\text{Business}} = 9.56\,\text{ms}, \quad W^{(A)}_{\text{Database}} = 14.15\,\text{ms}, \quad W^{(A)} = 32.47\,\text{ms}.$$

Strategy B — Speed up (M/M/1 at higher rate). Business is upgraded by 3× ($\mu'_3 = 375$ req/s, $s'_3 = 2.67$ ms, $\rho = 0.533$) and Database by 4× ($\mu'_4 = 333$ req/s, $s'_4 = 3$ ms, $\rho = 0.600$). Using (3):

$$W^{(B)}_{\text{Business}} = 5.71\,\text{ms}, \quad W^{(B)}_{\text{Database}} = 7.50\,\text{ms}, \quad W^{(B)} = 21.96\,\text{ms}.$$

Figure 7 compares per-stage and total latency for both strategies. Strategy B achieves a mean E2E latency 10.5 ms (32%) lower than Strategy A at equal cost.

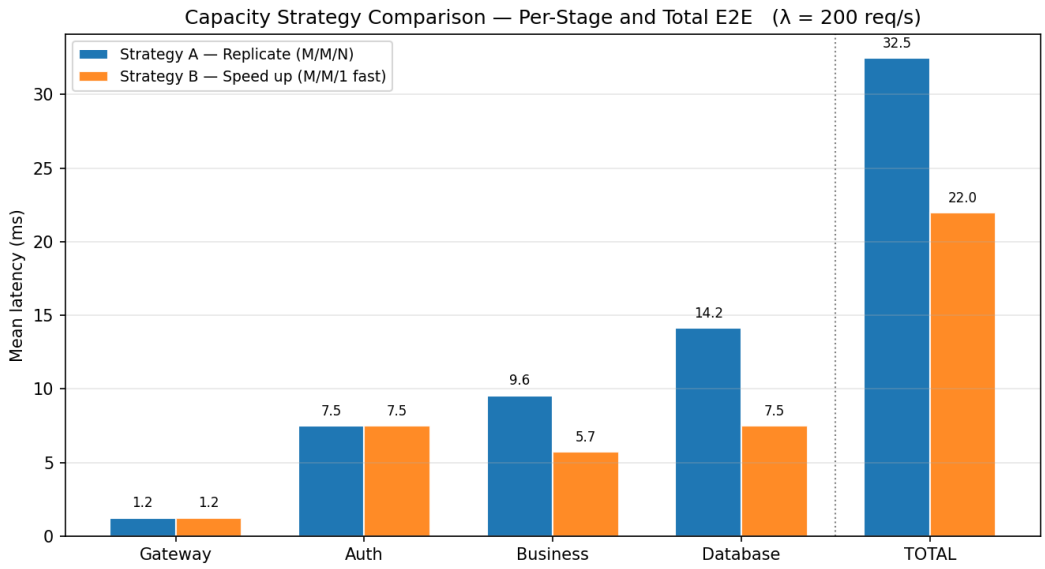

Figure 7: Per-stage and total E2E mean latency for Strategy A (replicate, M/M/N) and Strategy B (speed up, M/M/1 faster). Equal cost of 5 extra server units. Strategy B dominates in every stage and in total.

Why one fast server beats several slow ones. For a fixed offered load $a = \lambda/\mu$ and per-server utilisation $\rho = a/N$, the M/M/1 model with rate $N\mu$ (one fast server) has mean sojourn:

$$W^{(B)} = \frac{1}{N\mu(1-\rho)} = \frac{1}{N\mu - \lambda},$$

while the M/M/N model with N servers of rate μ has:

$$W^{(A)} = \frac{C(N,a)}{N\mu - \lambda} + \frac{1}{\mu}.$$

Since $C(N,a) \geq 0$ and $1/\mu > 0$, we always have $W^{(A)} \geq W^{(B)}$. The gap arises because N slower servers create coordination overhead: a request may arrive when all servers are busy even if their total capacity exceeds the load. A single fast server has no such overhead and also eliminates the need for a load balancer.

Figure 8 shows simulated empirical CDFs confirming the analytic result.

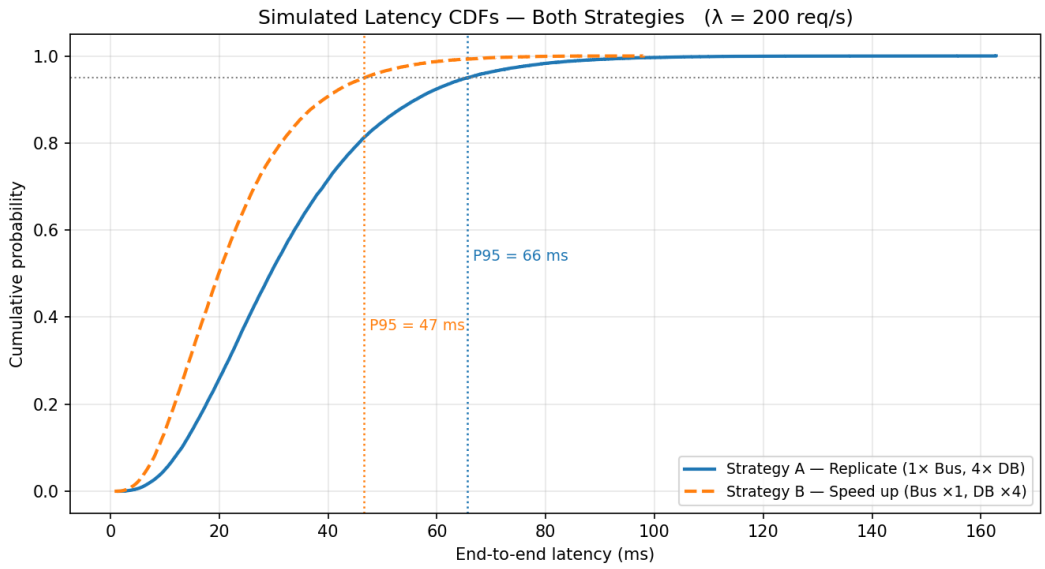

Figure 8: Empirical latency CDFs from simulation at $\lambda = 200$ req/s for both strategies (exponential service times). Strategy B (orange dashed) achieves lower P95 and a lighter tail than Strategy A (blue solid).

Recommendation. Adopt **Strategy B** (service-time reduction). At equal cost it reduces mean E2E latency by 32% and reduces P95 latency (as confirmed by simulation). Strategy A (replication) has the advantage of fault tolerance — a single server failure in Strategy B takes the stage offline — but this is an operational concern outside the scope of the latency model.

9 Critical Evaluation

9.1 Where the model succeeds

The M/M/1 / Jackson model works well for:

- Identifying the bottleneck stage analytically with zero computation (simply compare μ_i).
- Providing a closed-form lower bound on mean and P95 latency.
- Sensitivity analysis: the formula $W_i = s_i/(1-\rho_i)$ makes it immediate that a 10% reduction in s_4 at $\rho_4 = 0.8$ reduces W_4 by 33%, not 10%.
- Capacity planning ratios: the equal-cost comparison is tractable analytically.

9.2 Principal limitations

Exponential service-time assumption (A2). This is the dominant source of error. The lognormal simulation shows P95 is 178% higher than the analytic prediction at $\lambda = 50$ req/s. Database service times are bimodal in practice — cache hits settle around 1–2 ms while index misses stretch to 10–50 ms — giving CV $\gg 1$. The M/M/1 model is therefore a best-case lower bound on P95.

Independence assumption (A3). Jackson's theorem requires that no resource is shared between stages. In a real microservice deployment, all four services may run on the same Kubernetes node and share CPU, memory bandwidth, and network card. Under such conditions, a spike in one service can degrade all others simultaneously. The product-form model then over-predicts performance.

Open-loop model (A5). The model assumes requests arrive regardless of system state. Under overload, real clients implement timeouts and retries. Retried requests increase the effective arrival rate, which creates a positive feedback loop that can cause cascading failure. The model cannot predict or represent this regime.

No spatial or temporal heterogeneity. The model uses a single mean service time per stage. Real systems have diurnal load patterns, version-dependent service rates, and cache warming effects. A single operating point is insufficient for capacity planning over a full day.

9.3 Next steps

1. **Measure service-time CVs in production.** The most useful single measurement is the empirical CV of Database service times. Even a rough estimate ($\pm 20\%$) would let the lognormal simulation be calibrated to real data rather than the assumed $\mathrm{CV} = 2$.
2. **Extend to G/G/1 (Kingman's approximation).** For general arrival and service processes, Kingman's formula provides a closed-form approximation:

$$W_q \approx \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \cdot \frac{1}{\mu},$$

   where c_a, c_s are the CVs of inter-arrival and service times. This would replace the exponential assumption with measured CVs.
3. **Model shared-resource contention.** If stages co-locate on the same host, a coupled model (closed queueing network or simulation with shared CPU pool) is needed.
4. **Add timeout and retry logic.** A non-linear ODE or Markov chain model capturing retry amplification would reveal the actual collapse point under overload.

9.4 What I would not trust this model for

- Predicting P99 or higher percentiles (the tail is exponential under M/M/1 but power-law in practice).
- Planning for flash-crowd or correlated arrival events.
- Making deployment decisions without validating service-time CVs against production traces.

Use of AI Tools

AI tools — including ChatGPT (for brainstorming and concept review), Grammarly (for proofreading), and GitHub Copilot (for debugging assistance) — were used during the preparation of this work. Their use was limited to supporting study, generating draft explanations for review, and identifying syntax errors in code. All modelling decisions, derivations, analysis, and conclusions are the author's own.

Reproducibility Statement

All results are produced by running `python main.py` from the `Problem 13/` directory with `requirements.txt` satisfied. The analytic code is fully deterministic. The simulation uses `numpy.random.default_rng(seed)` with seeds 1000–1007; the base seed is declared in `simulation.py` line 32.

References

[1] J. R. Jackson, "Networks of Waiting Lines," *Operations Research*, vol. 5, no. 4, pp. 518–521, 1957.

[2] G. Latouche and V. Ramaswami, *Introduction to Matrix Analytic Methods in Stochastic Modeling*. SIAM, Philadelphia, 1999.

[3] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems*. Cambridge University Press, 2013.

[4] J. D. C. Little, "A Proof for the Queuing Formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[5] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[6] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, New York, 1975.